Тема 6. Типы данных нелинейной структуры

Лекции 14

Графы

Цели и задачи изучения темы

Целью данной темы является изучение типов данных нелинейной структуры, таких как графы и деревья. В ней рассматриваются основные понятия и определения теории графов, способы представления графов в памяти ЭВМ, алгоритмы обхода графов, основные понятия и определения деревьев, широко распространенный частный случай деревьев – двоичные деревья и их применение, структуры данных для представления и операции над двоичными деревьями.

6.2 Алгоритмы обхода графов

Для прохождения нелинейных структур требуется разработать стратегию доступа к узлам и маркирования узлов после обработки.

Большинство алгоритмов на графах имеют в основе такой систематический перебор вершин графа такой, что каждая вершина просматривается один раз.

Среди алгоритмов обхода графов выделяют два принципиально различных подхода:

- Поиск в глубину;
- Поиск в ширину.

6.3.1. Поиск в глубину

Начальная вершина передается в качестве параметра и становится первой обрабатываемой вершиной. По мере продвижения вниз до тупика смежные вершины запоминаются в стеке. Если еще остались необработанные вершины, то осуществляется возврат к вершинам в стеке и продолжается поиск по другому пути. Обработанные вершины образуют множество всех вершин, достижимых из начальной вершины.

Для хранения обработанных вершин используется список *L*, а для запоминания смежных вершин – стек *S*. Приведем алгоритм поиска в глубину:

1. Начальная вершина помещается в стек.
2. Итерационный процесс выталкивания вершины из стека и ее обработки:
    a) вытолкнуть вершину *V* из стека;
    b) проверить по списку *L*, была ли она обработана;
    c) если нет, то произвести обработку этой вершины и получить список смежных с ней вершин;
    d) включить *V* в список *L*, чтобы избежать повторной обработки;
    e) поместить в стек те смежные с *V* вершины, которых еще нет в списке *L*.
3. Когда стек становится пустым, процесс завершается и возвращает список отработанных вершин.

Рассмотрим пример применения этого алгоритма. Пусть дан граф, представленный на рис. 6.6.

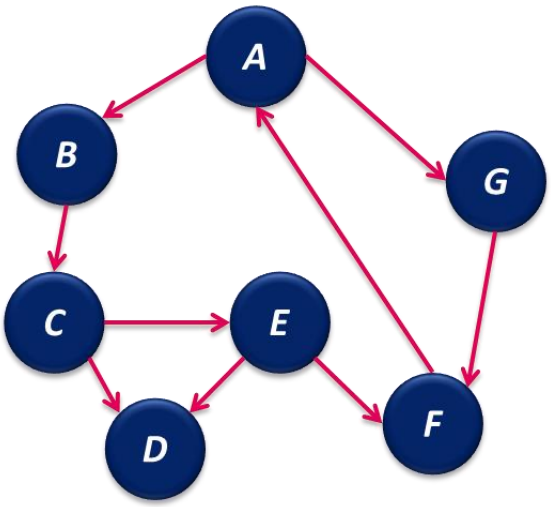


Рис. 6.6. Пример графа для иллюстрации поиска в глубину и ширину

Шаг 1. Вершина *A* является начальной и помещается в стек.

Шаг 2. Вершина *A* выталкивается из стека и обрабатывается. Вершина *A* включается в список обработанных вершин, смежные с ней вершины *B* и *G* помещаются в стек.

Шаг 3. Вершина *G* выталкивается из стека. Так как этой вершины пока нет в списке *L*, она включается в него. В стек помещается вершина *F*, смежная с *G*.

Шаг 4. Вершина *F* выталкивается из стека и помещается в список *L*. Смежная с ней вершина *A* уже есть в списке.

Шаг 5. Обработка вершины *B*.

Шаг 6. Обработка вершины *C*.

Шаг 7. Вершина *E* имеет две смежные вершины *D* и *F*. Вершина *F* имеется в списке *L* и поэтому пропускается. Вершины *D* в списке нет, поэтому она помещается в стек.

Шаг 8. Поиск завершается после обработки вершины *D*. Второй экземпляр *D* в стеке игнорируется, так как эта вершина уже находится в списке *L*.

Рис. 6.7 иллюстрирует описанную выше последовательность действий.

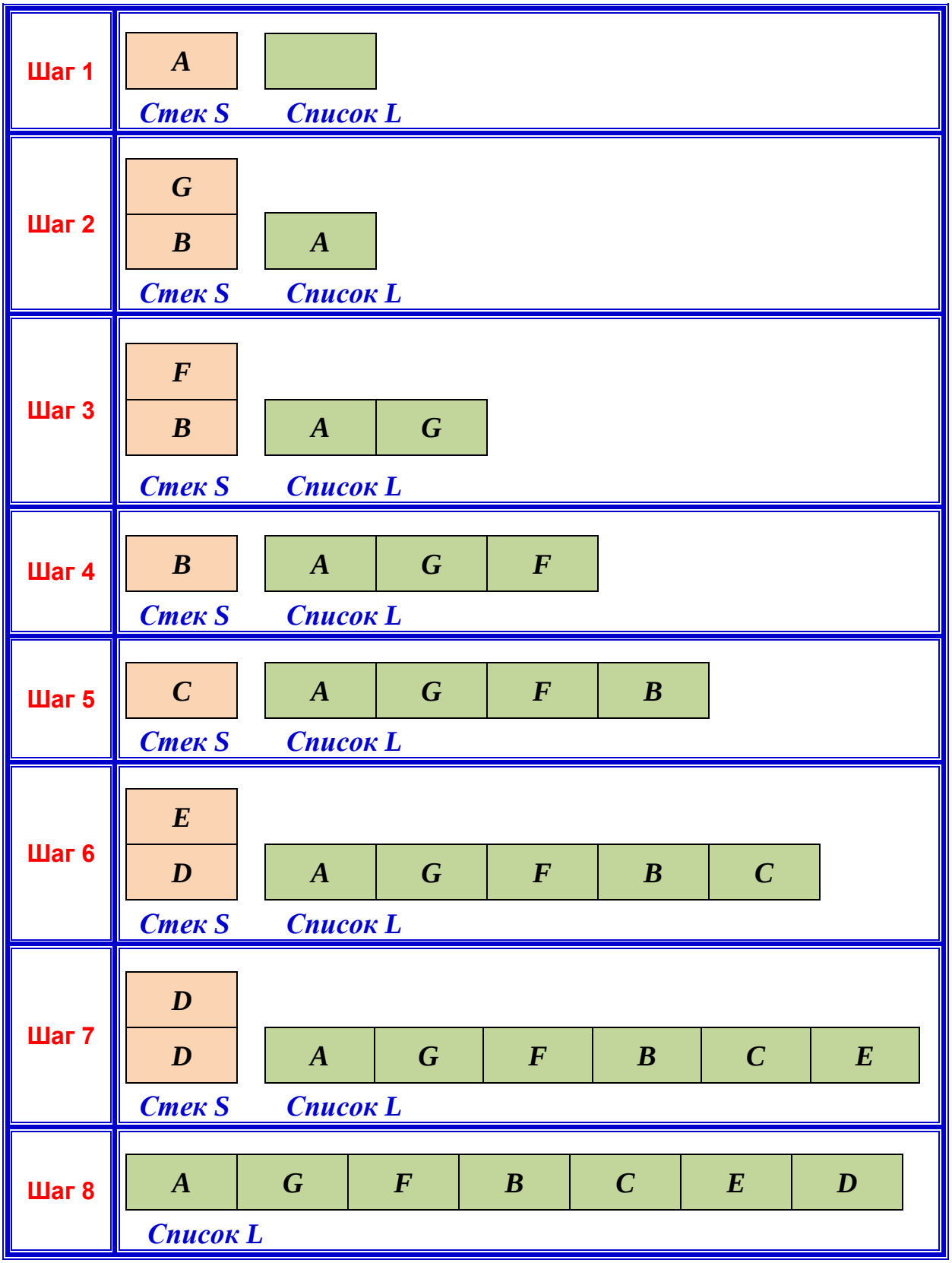


Рис. 6.7. Пример поиска в глубину для графа на рис. 6.6

6.3.2. Поиск в ширину

Поиск в ширину начинает работу с некоторой начальной вершины и затем производит обработку каждой вершины, смежной с ней. Затем сканирование продолжается на следующем уровне смежных вершин, и т.д. до конца пути.

Для хранения обработанных вершин используется список *L*, а для запоминания смежных вершин – очередь *Q*. Приведем алгоритм поиска в ширину:

1. Начальная вершина помещается в очередь.
2. Итерационный процесс удаления вершины из очереди и ее обработки:
   a) удалить вершину *V* из очереди;
   b) проверить по списку *L*, была ли она обработана;
   c) если нет, то произвести обработку этой вершины и получить список смежных с ней вершин;
   d) включить *V* в список *L*, чтобы избежать повторной обработки;
   e) поместить в очередь те смежные с *V* вершины, которых еще нет в списке *L*.
3. Когда очередь становится пустой, процесс завершается и возвращает список отработанных вершин.

Рассмотрим пример применения этого алгоритма. Пусть дан граф, представленный на рис. 6.6.

Шаг 1. Вершина *A* является начальной и помещается в очередь.

Шаг 2. *A* удаляется из очереди и обрабатывается. *A* включается в список обработанных вершин, смежные с ней вершины *B* и *G* помещаются в очередь.

Шаг 3. Вершина *B* удаляется из очереди. Так как этой вершины пока нет в списке *L*, она включается в него. В очередь помещается вершина *C*, смежная с *B*.

Шаг 4. Вершина *G* удаляется из очереди и помещается в список *L*. Смежная с ней вершина *F* помещается в очередь.

Шаг 5. Обработка вершины *C*.

Шаг 6. Обработка вершины *F*.

Шаг 7. Обработка вершины *D*.

Шаг 8. Обработка вершины *E*.

Рис. 6.8 иллюстрирует описанную выше последовательность действий.

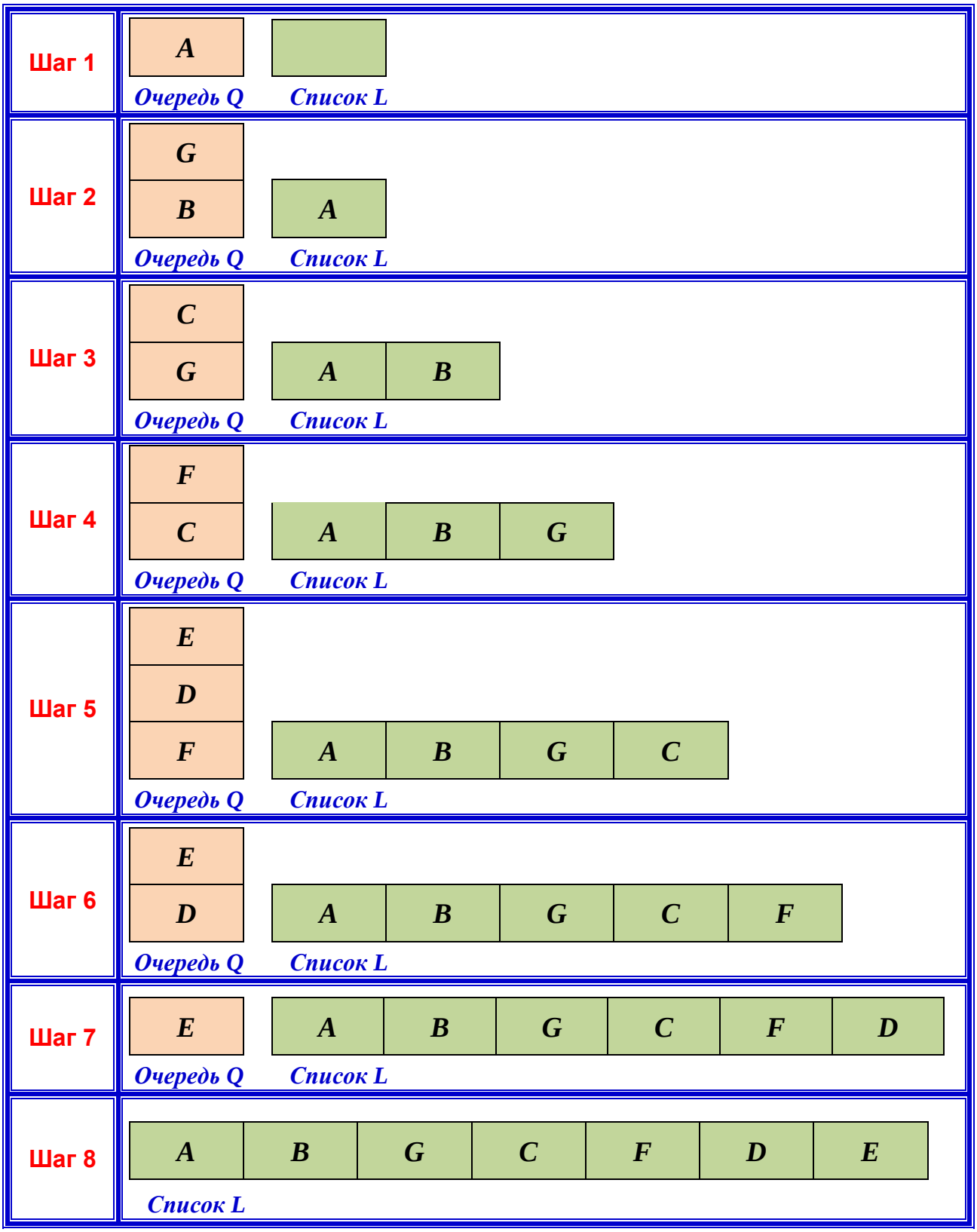


Рис. 6.8. Пример поиска в ширину для графа на рис. 6.6